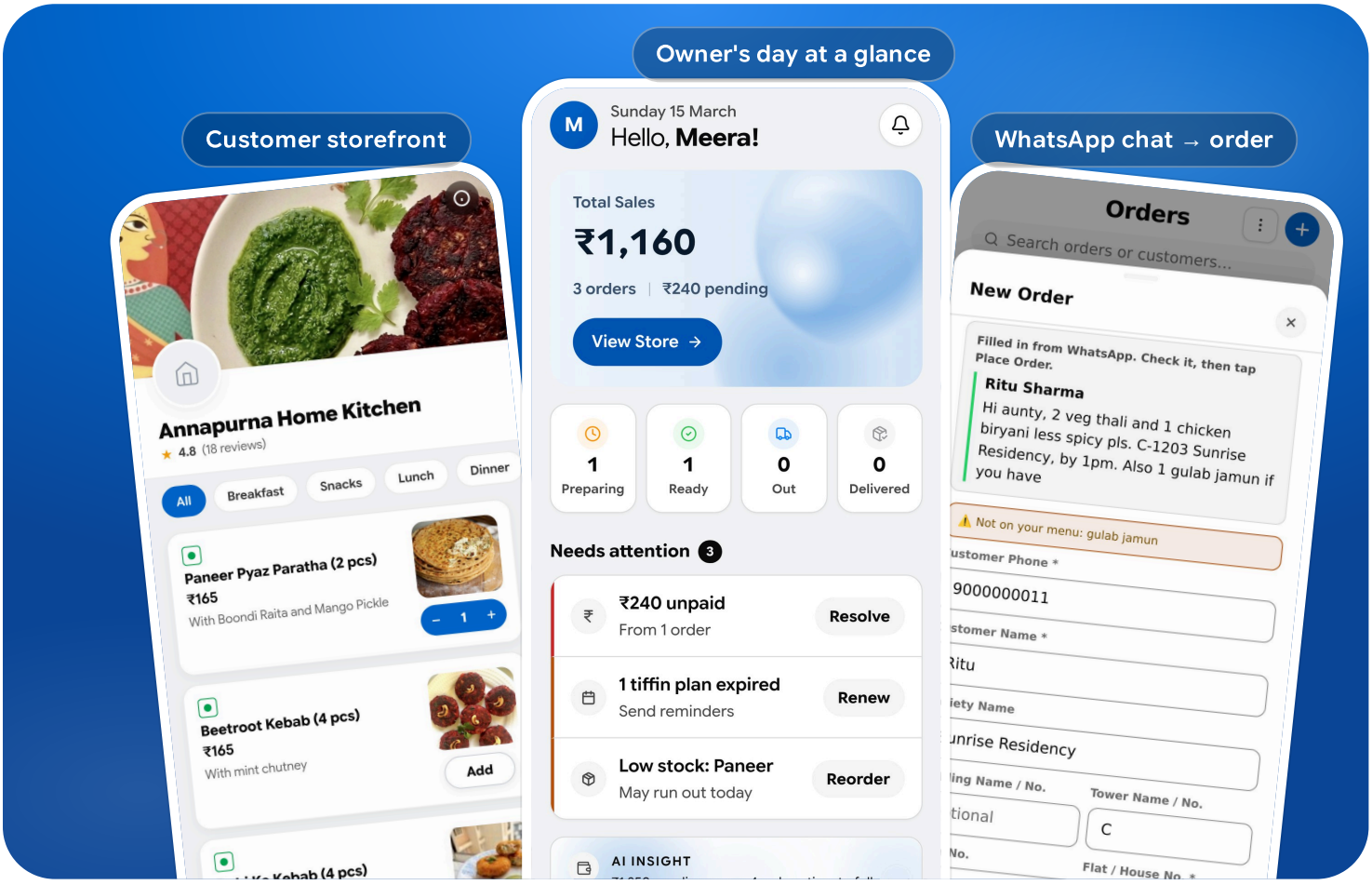
OrdersIn

The order book that already speaks WhatsApp.

A live mobile web app that runs the daily operations of a one-person food business, from orders and billing to payments, stock and customers, without asking anyone to change how they already work.

Live in supervised beta with a real home kitchen · English · Hindi · Kannada



WHAT FOLLOWS

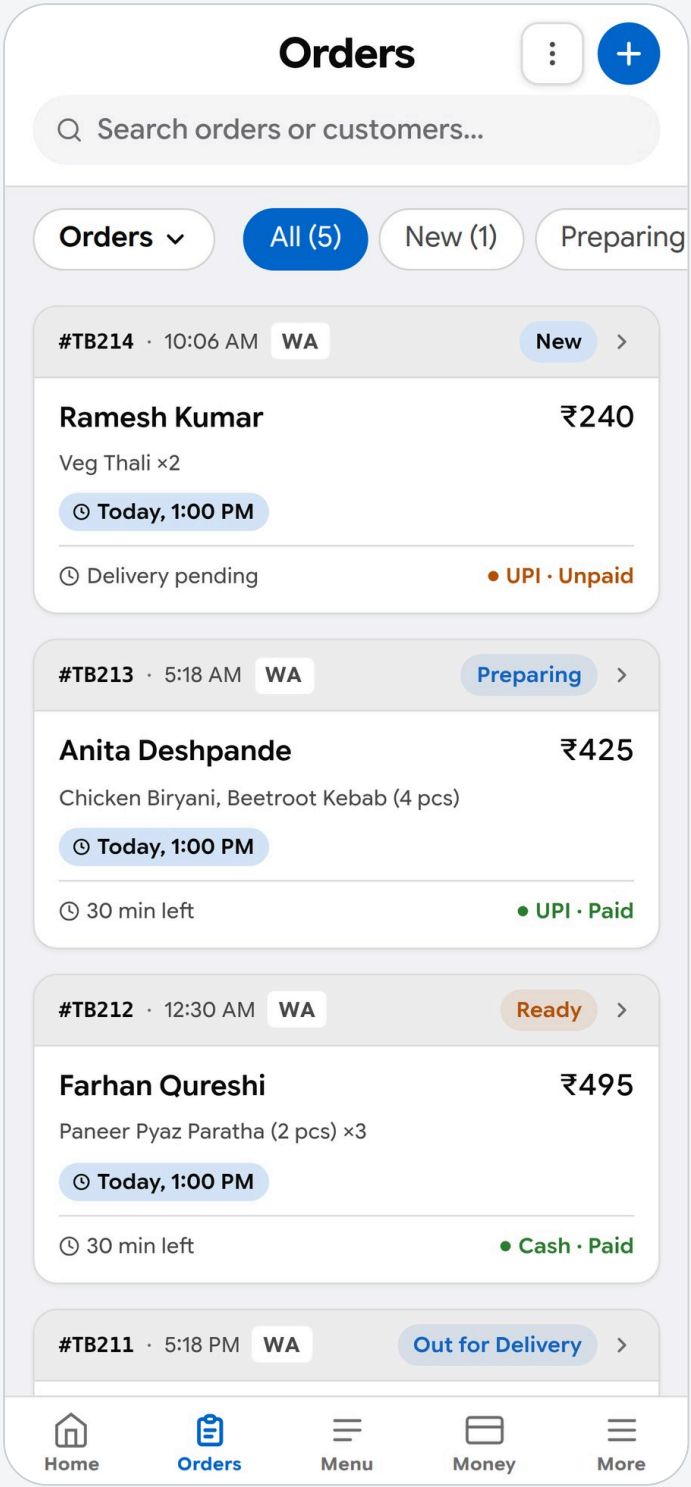
1–4	The core loop
5	The customer side
6–8	The daily dashboard
9–11	Money
12–14	Business insight
15	Kitchen Assistant
16–20	Catalogue, stock and data
21	In testing: orders straight from WhatsApp
22	Under the hood
	React 19 · TypeScript · Firebase · PWA

Screens from the current production build. Most use a demo kitchen with fictional data. The menu is from the live beta kitchen, with names and payment details redacted; the storefront borrows a few of its dish photos.

Built and directed by **Nitin Rawal** · ordersin.co/app

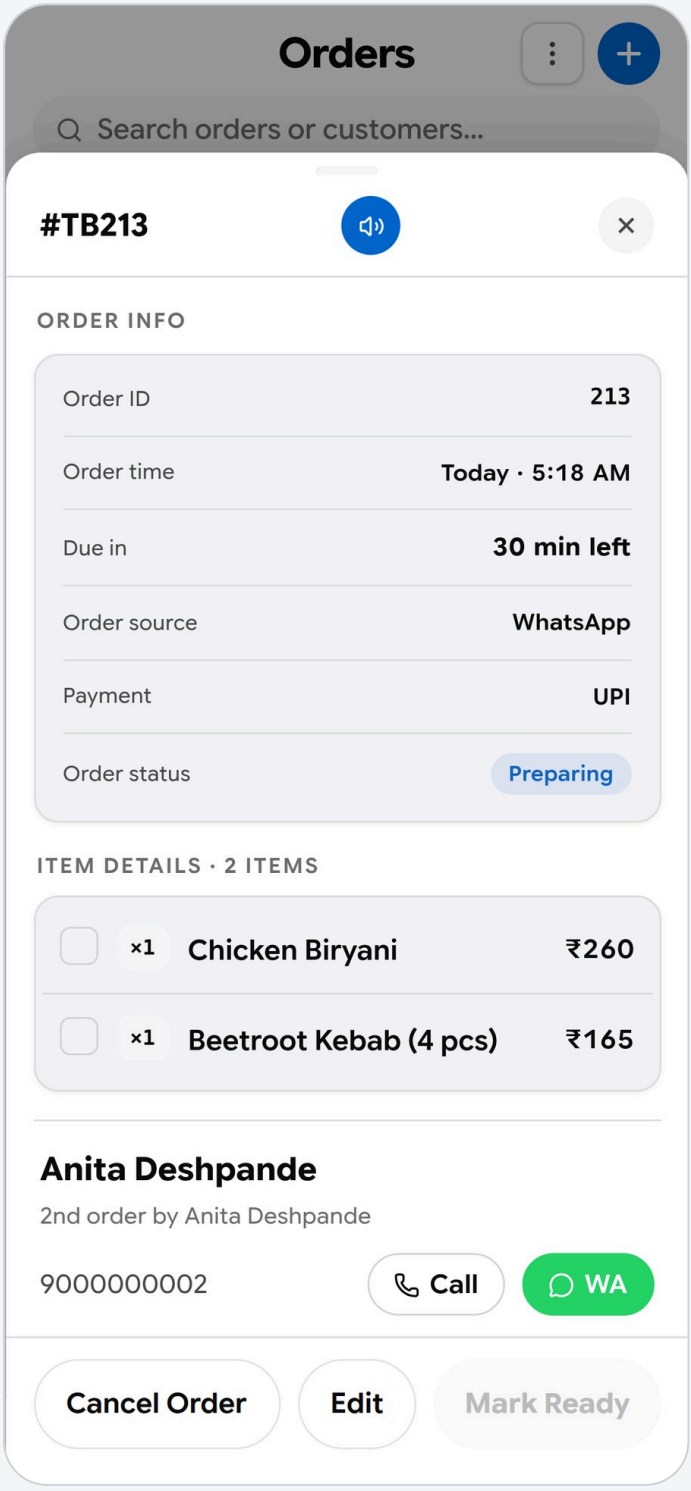
Orders

The live order list. Each card shows its source (WA = WhatsApp), its status, and whether the money has been collected.



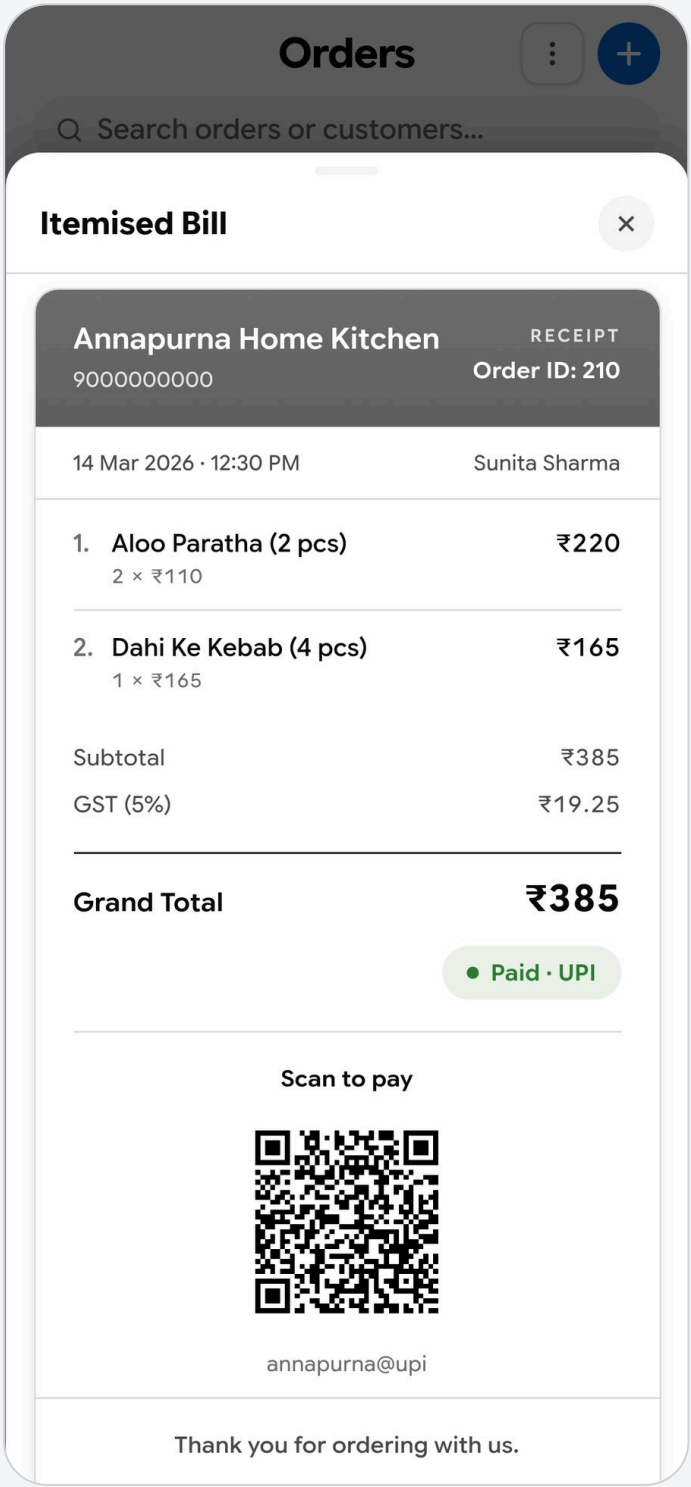
Order ticket

One order in full: items, customer, address, one-tap Call and WhatsApp. The speaker reads the ticket aloud in the owner's language.



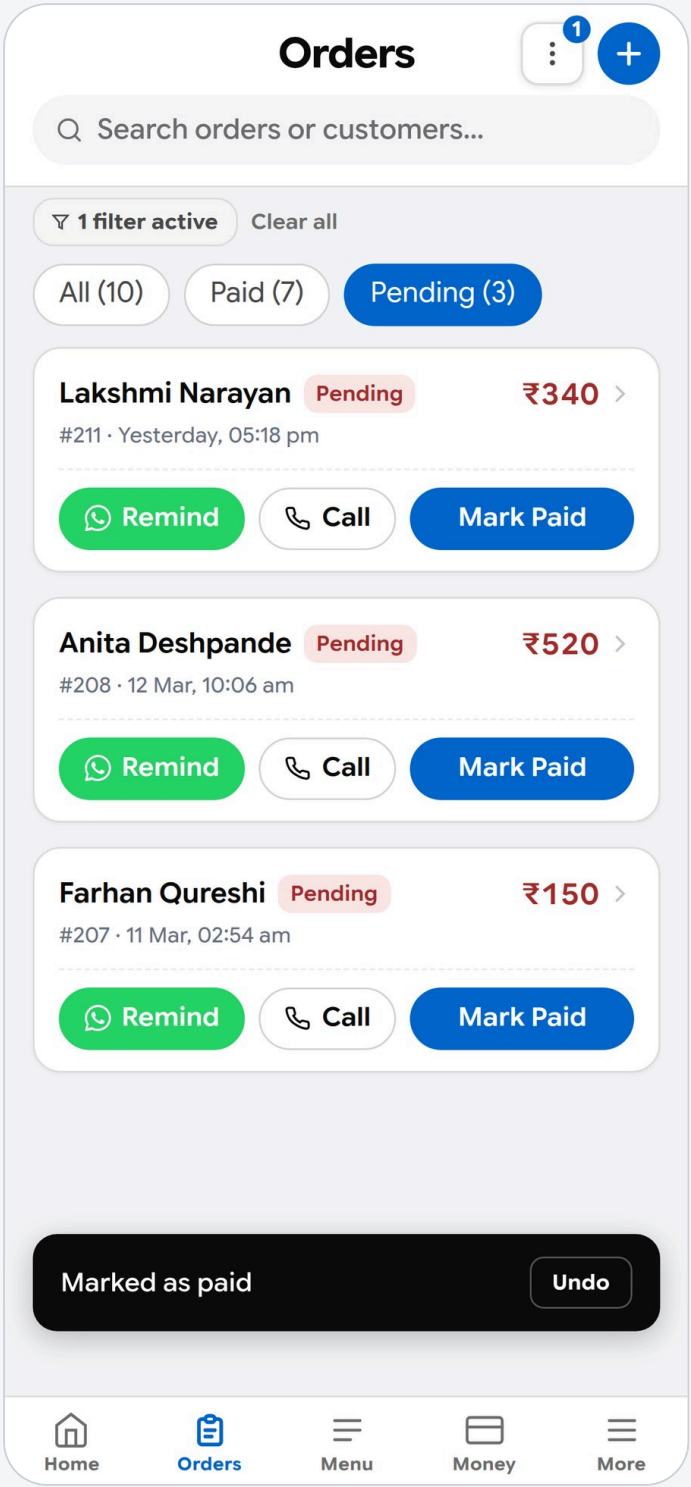
Itemised bill

Generated in one tap with a UPI QR code the customer can scan, then sent back on WhatsApp.



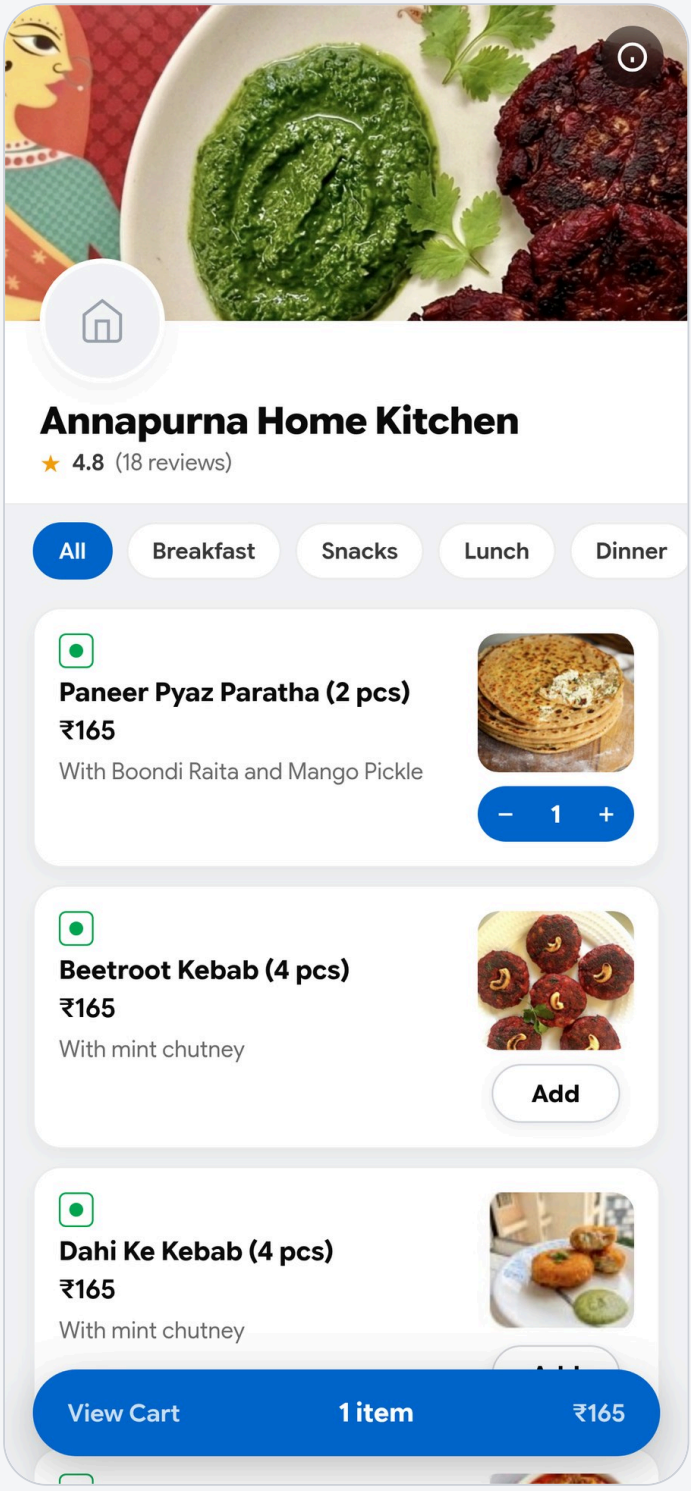
Marked paid

The owner confirms the payment and dues update everywhere, with a few seconds to undo.



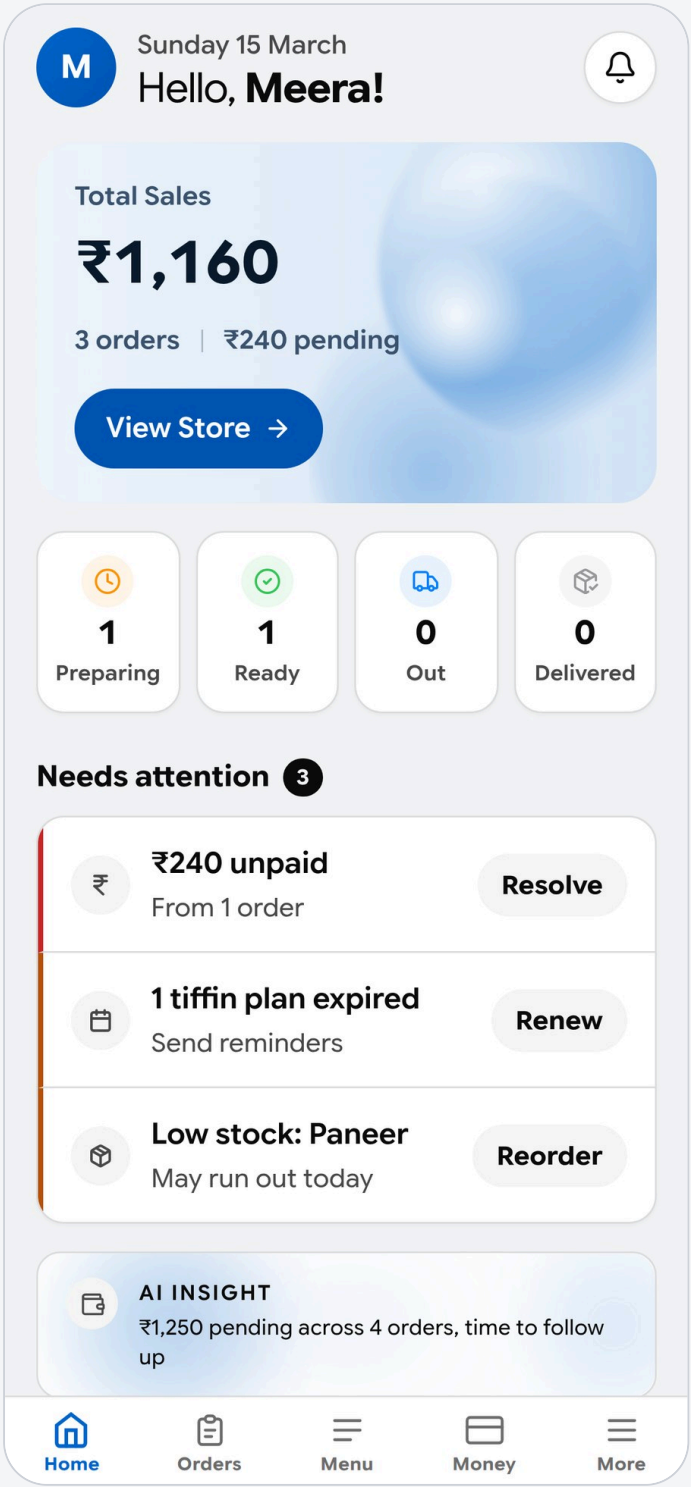
Customer storefront

Customers order from a link the kitchen shares. No app install, no account. Orders drop straight into the owner's list.



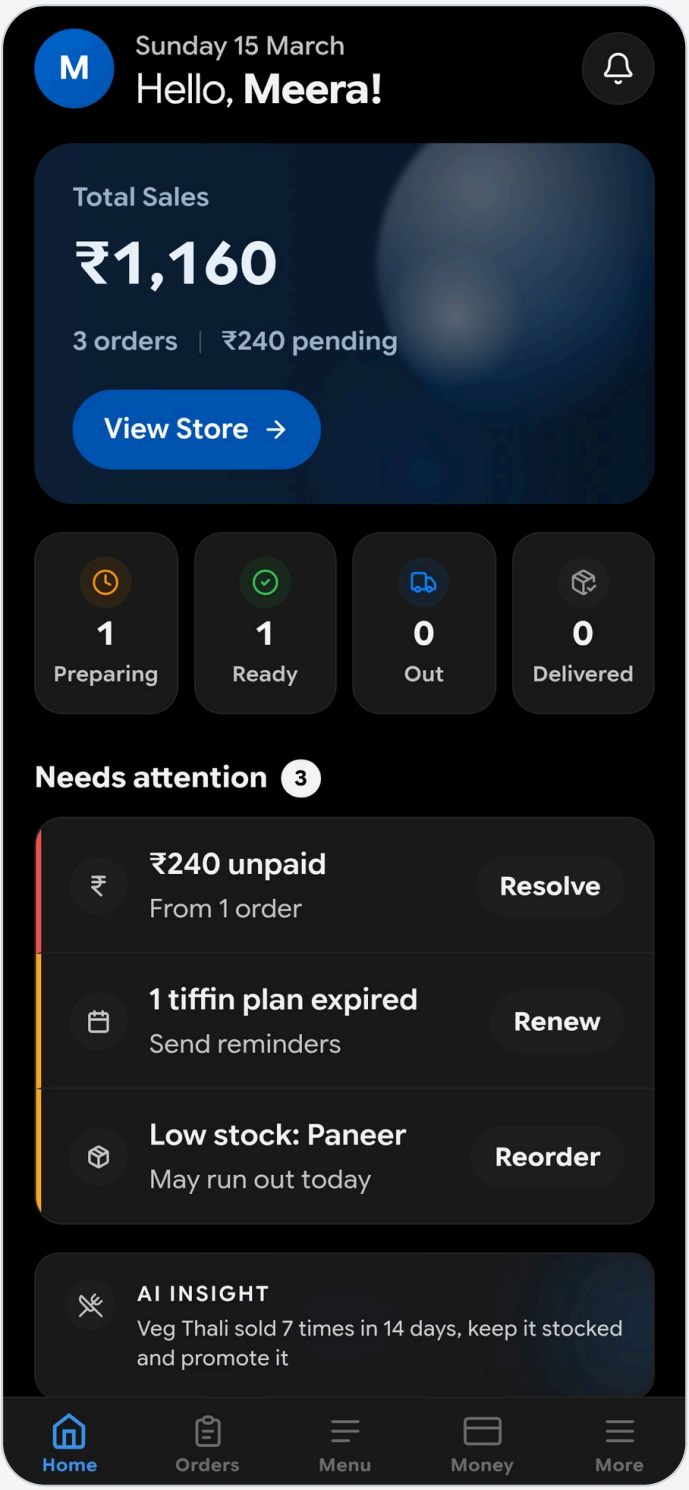
Home

Today’s sales, order states, and what needs attention: dues, expiring tiffin plans, low stock.



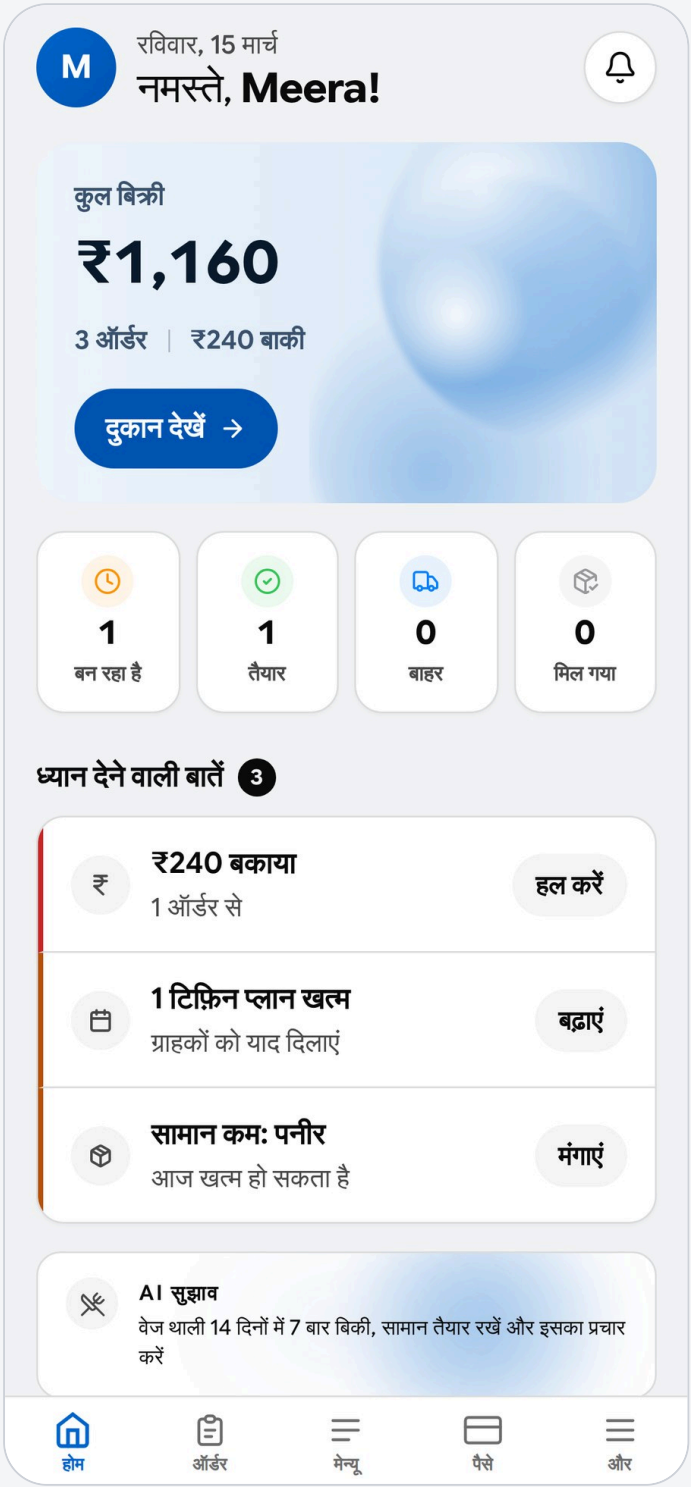
Home in dark mode

A full dark theme across every screen.



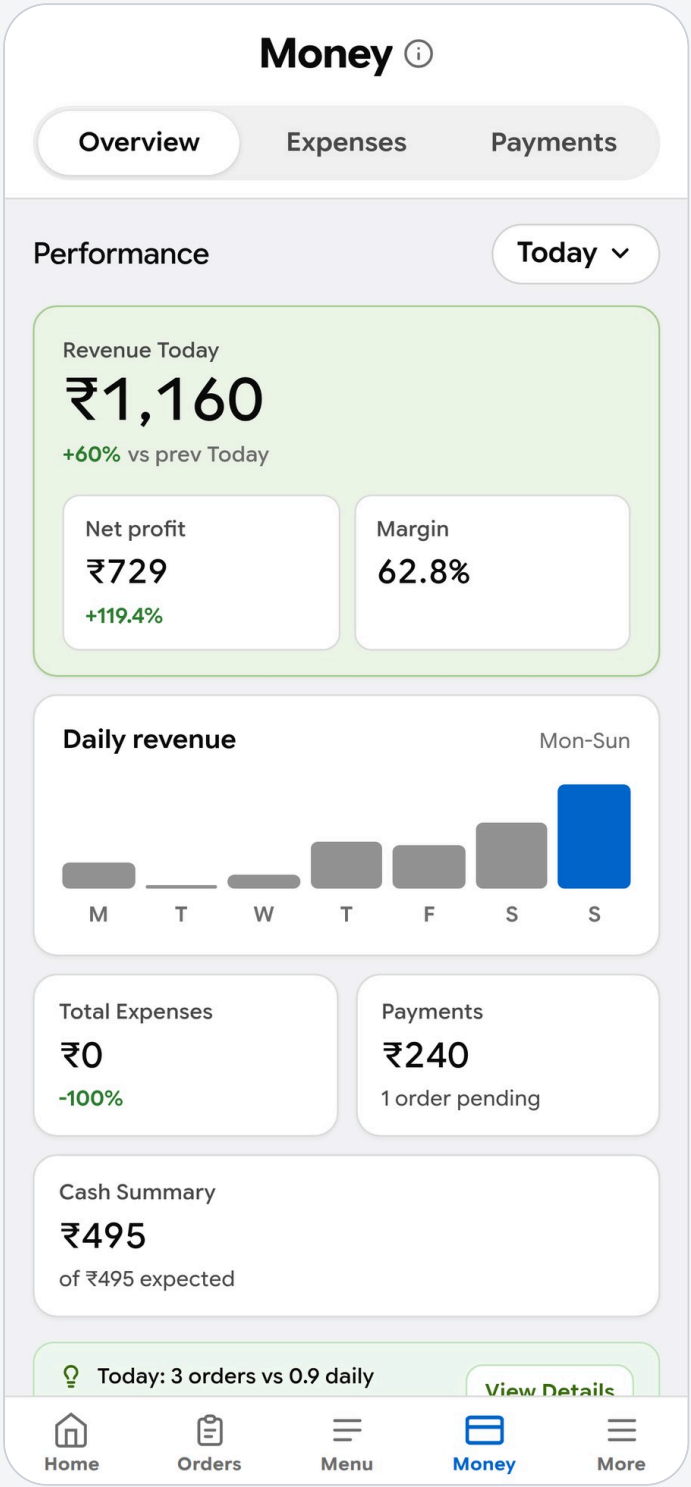
Home in Hindi

A complete UI translation in English, Hindi and Kannada, about 1,650 strings each, checked by an automated coverage script.



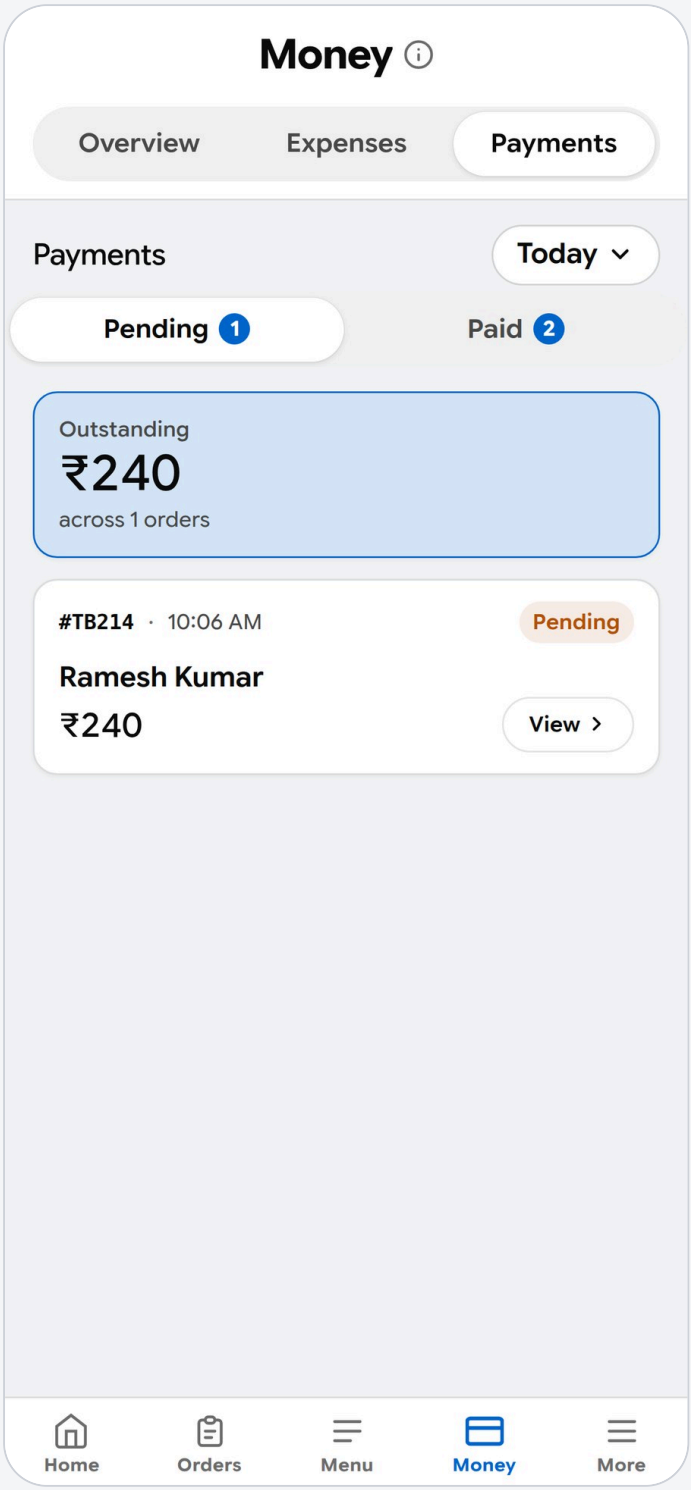
Money

Revenue, net profit and margin. Profit is revenue minus dish cost, packaging and delivery, not revenue in disguise.



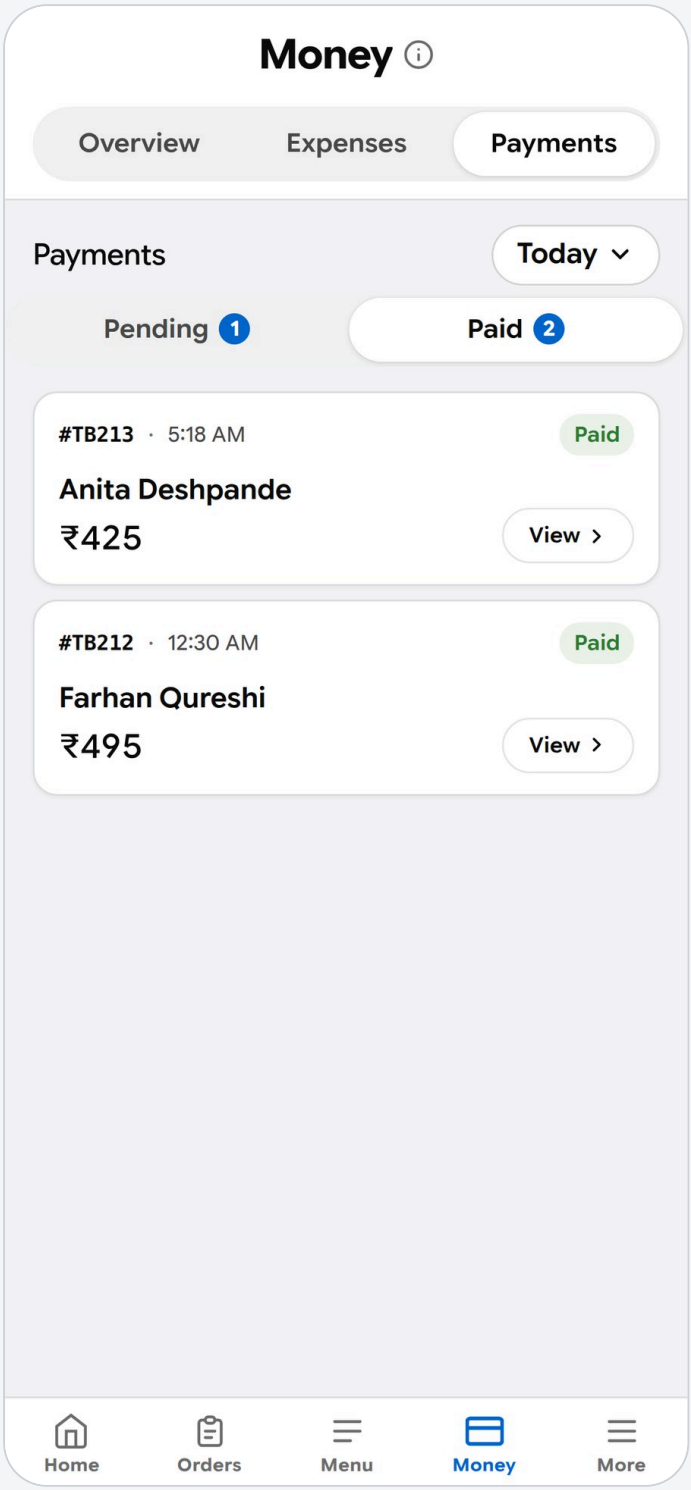
Pending payments

What is still owed, across every order, at a glance.



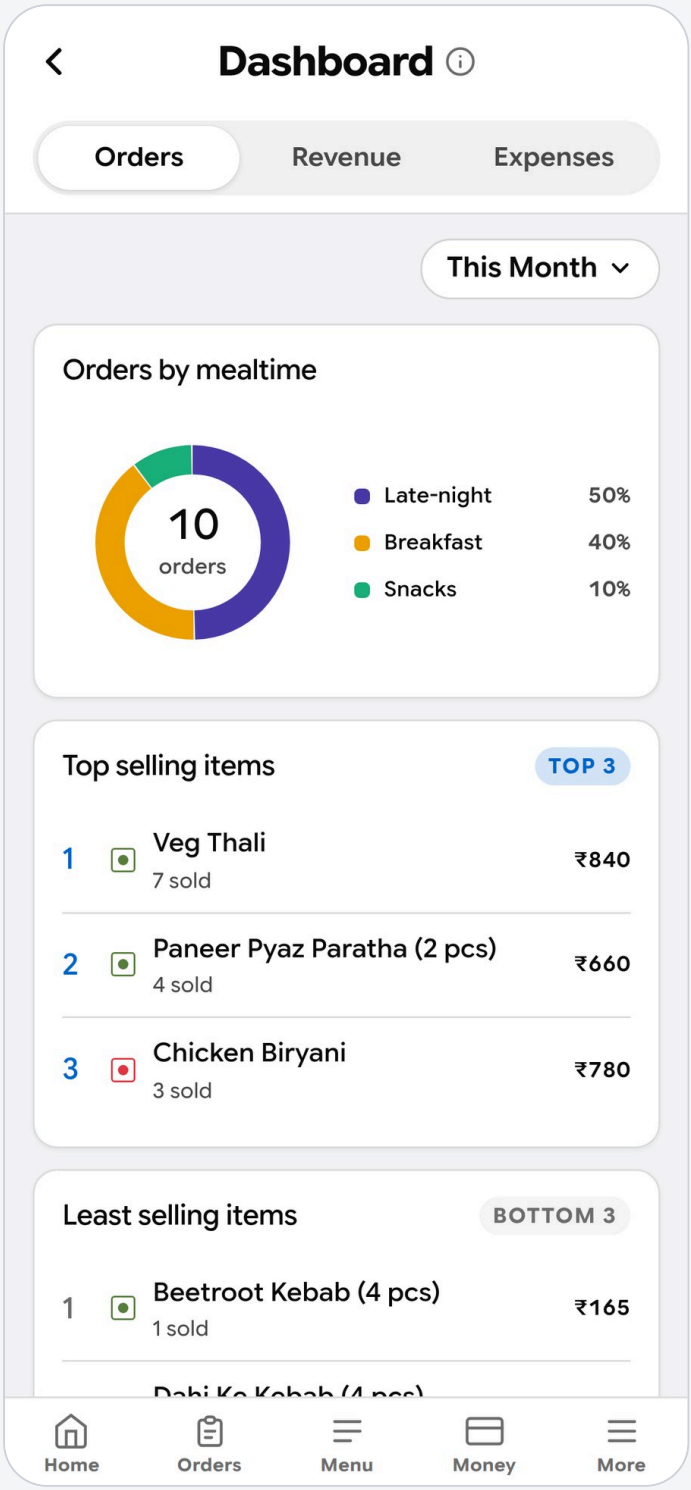
Payment history

Every settled payment, by order.



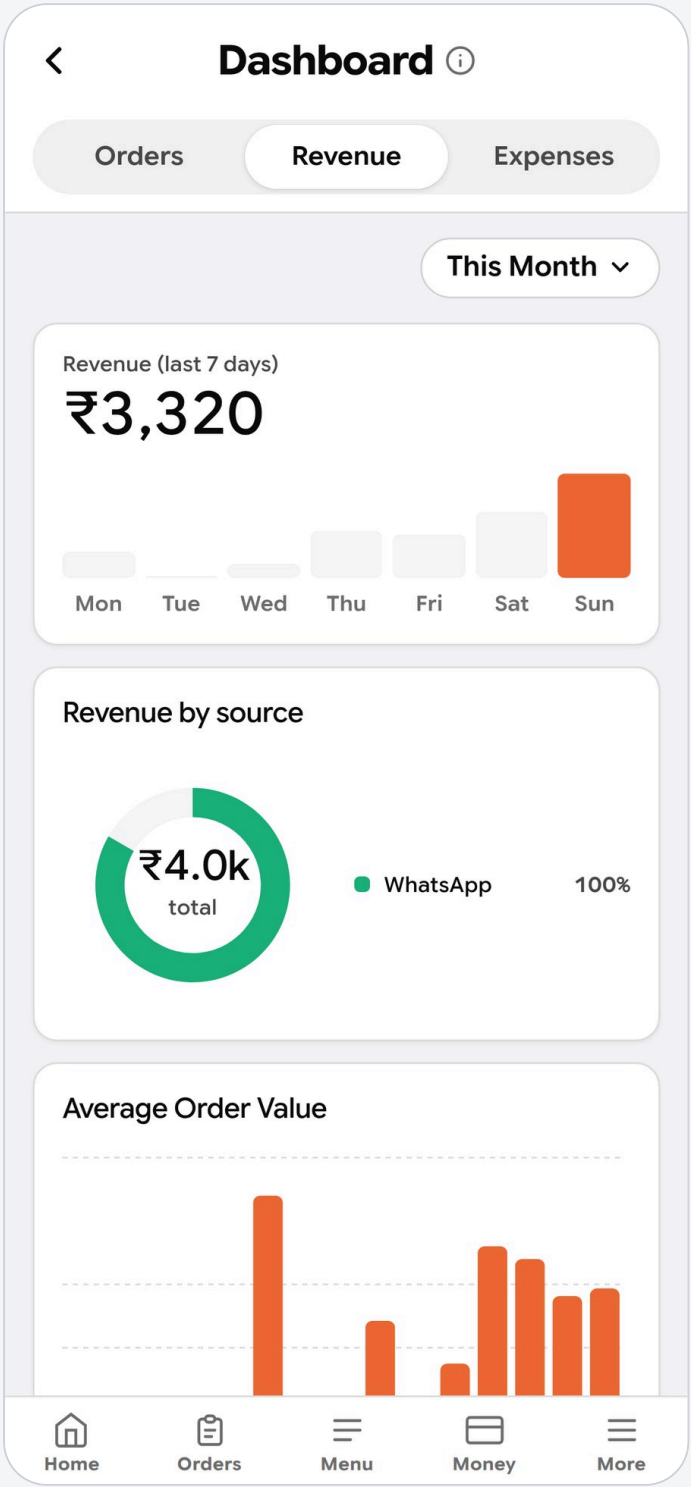
Insights: orders

Orders by mealtime, and the best and worst selling dishes.



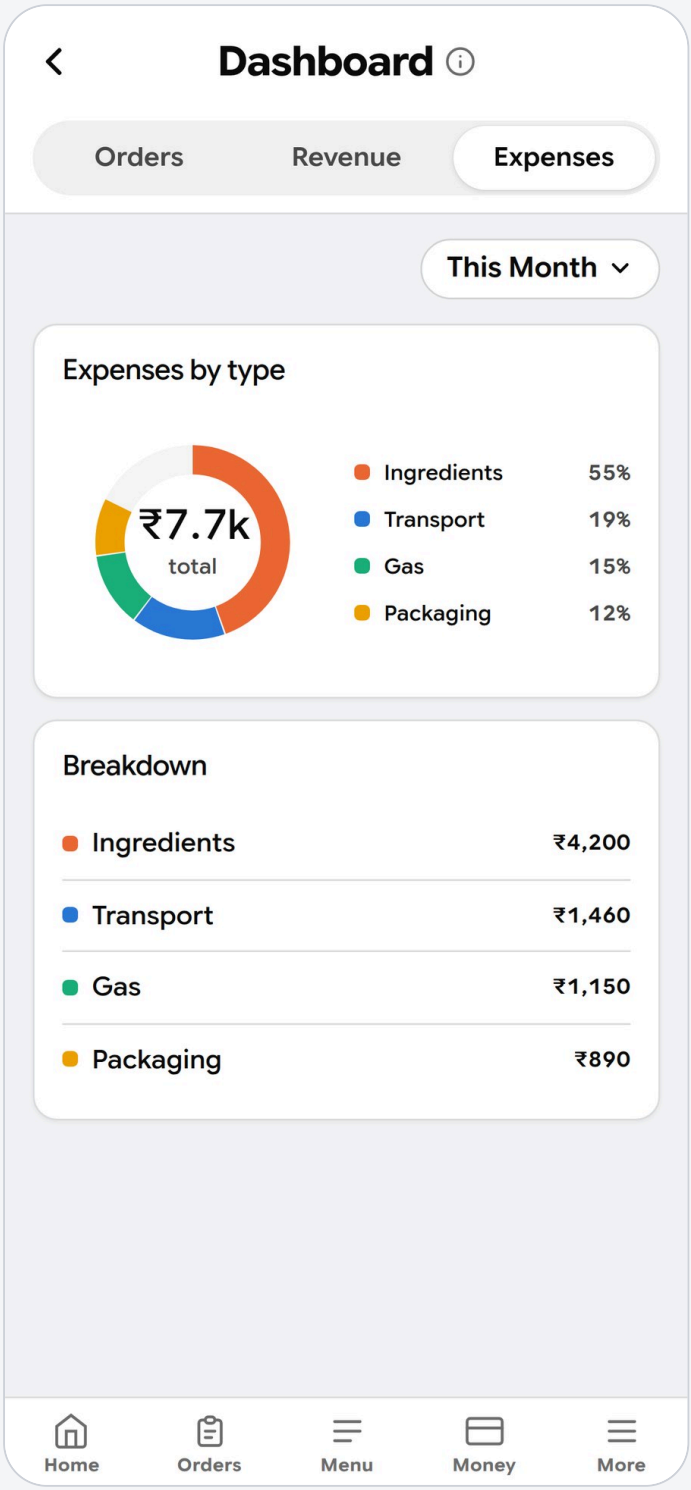
Insights: revenue

Revenue by source. For the beta kitchen, 100% arrives through WhatsApp, which is why the app is built around it.



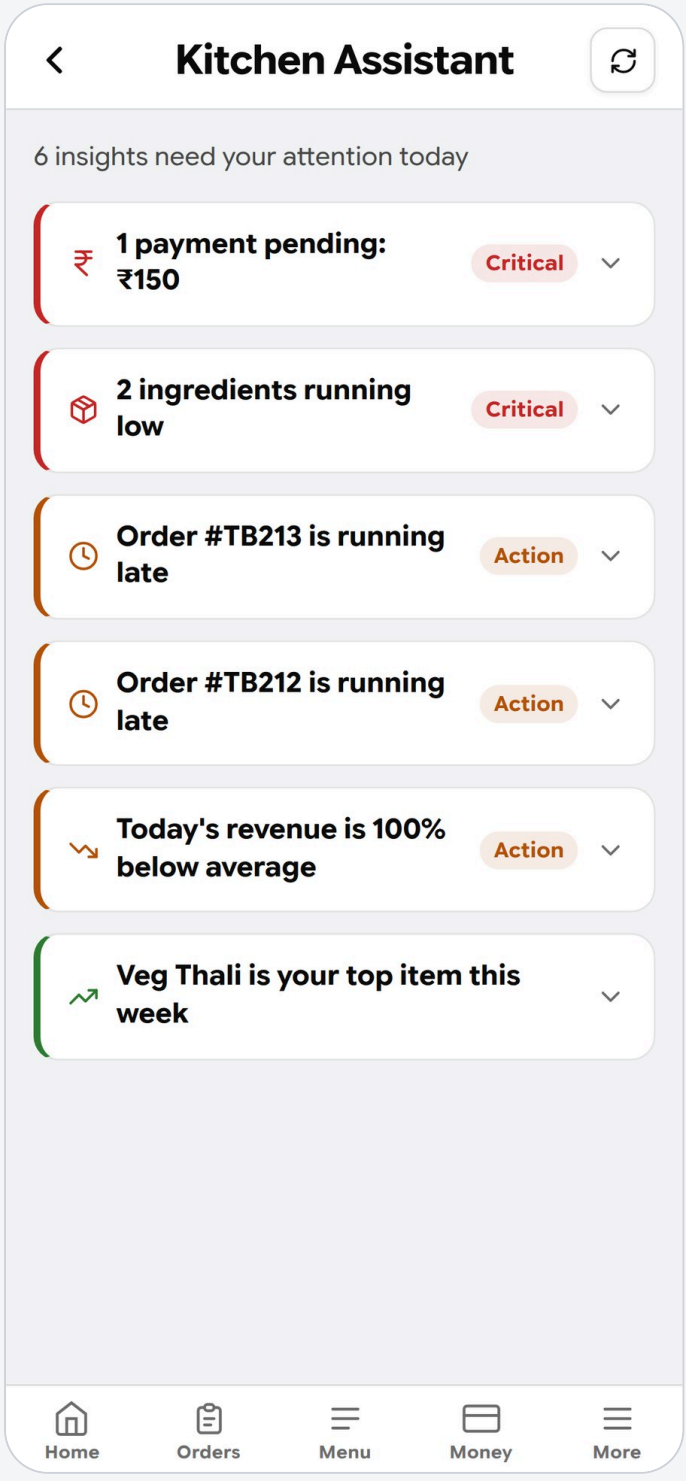
Insights: expenses

Where the money goes, split by type.



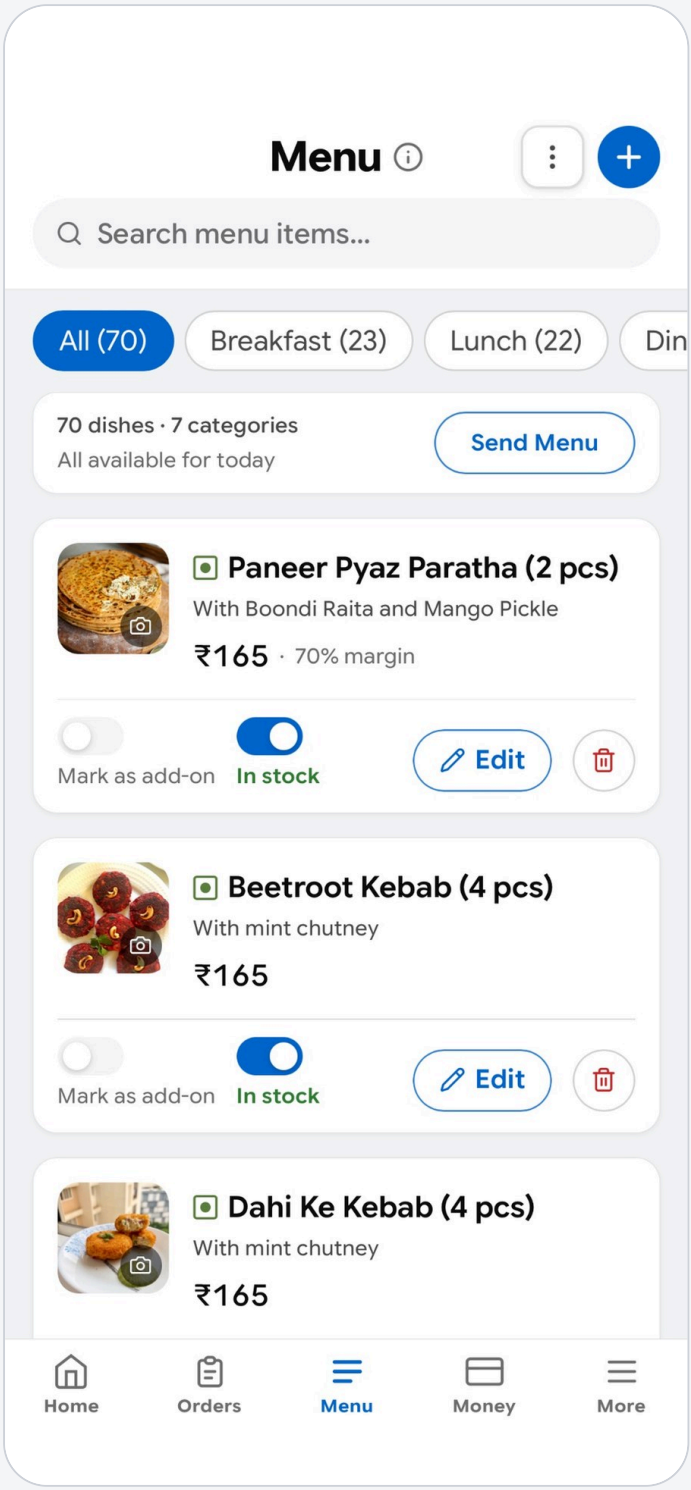
Kitchen Assistant

A rules engine (8 checks) over the kitchen's live data: low stock, late orders, unpaid dues, each with an action attached. Deterministic and explainable: no AI model, no inference cost.



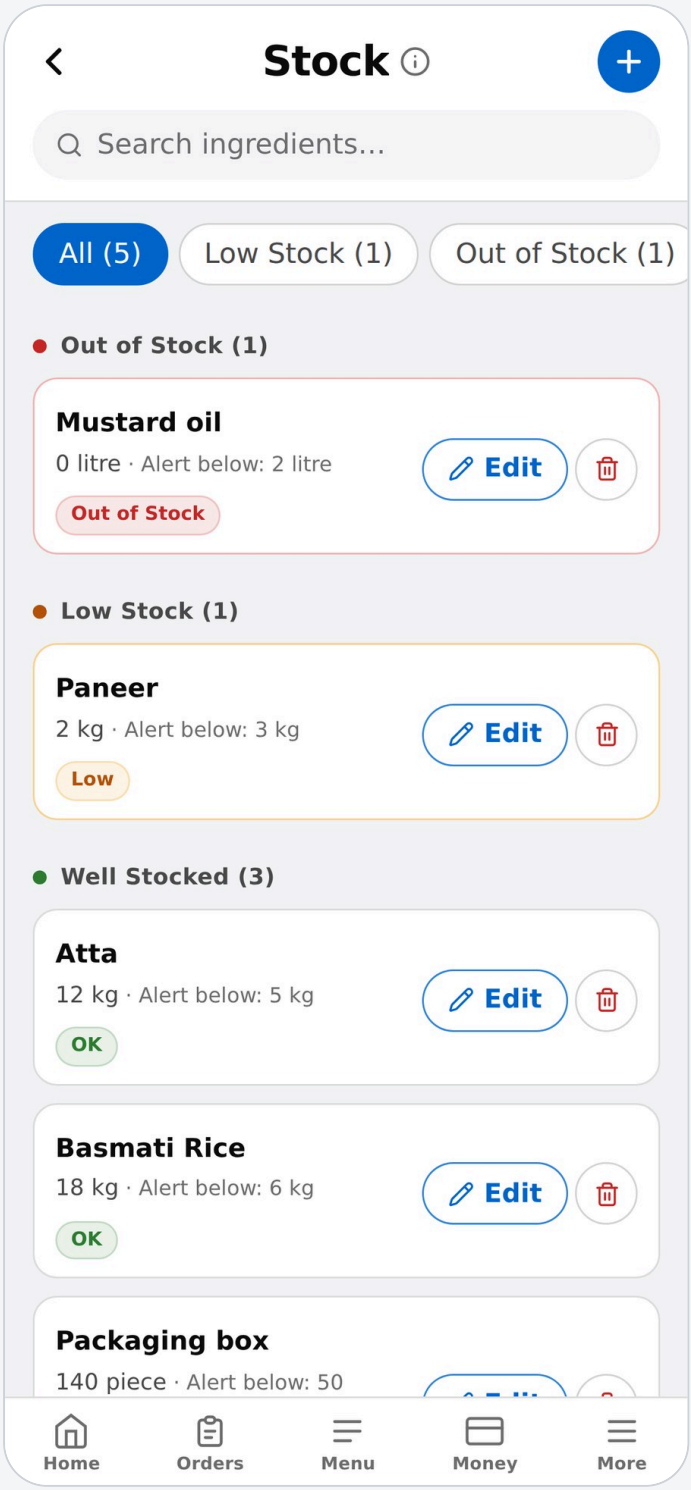
Menu

70 dishes across 7 categories, with photos, prices and per-dish margin.



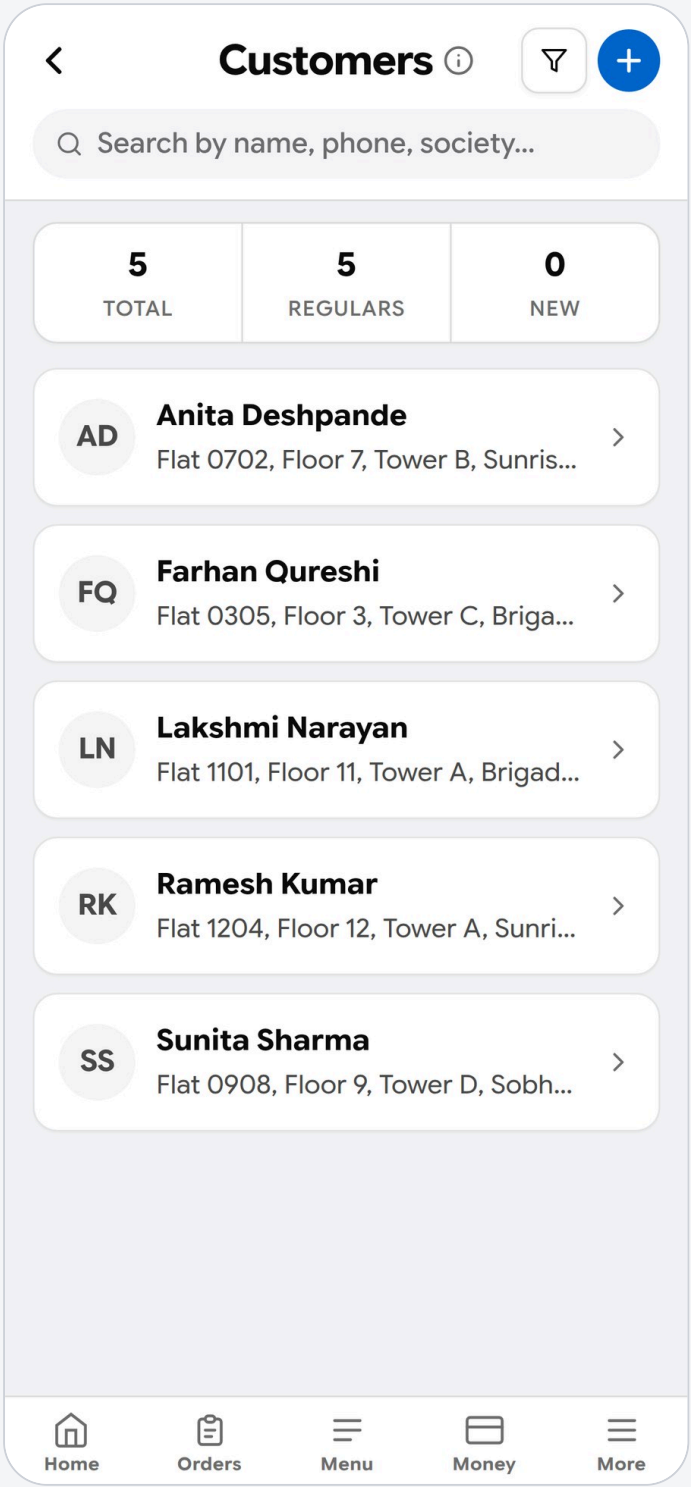
Stock

Ingredient levels with per-item reorder thresholds, depleting as orders go out.



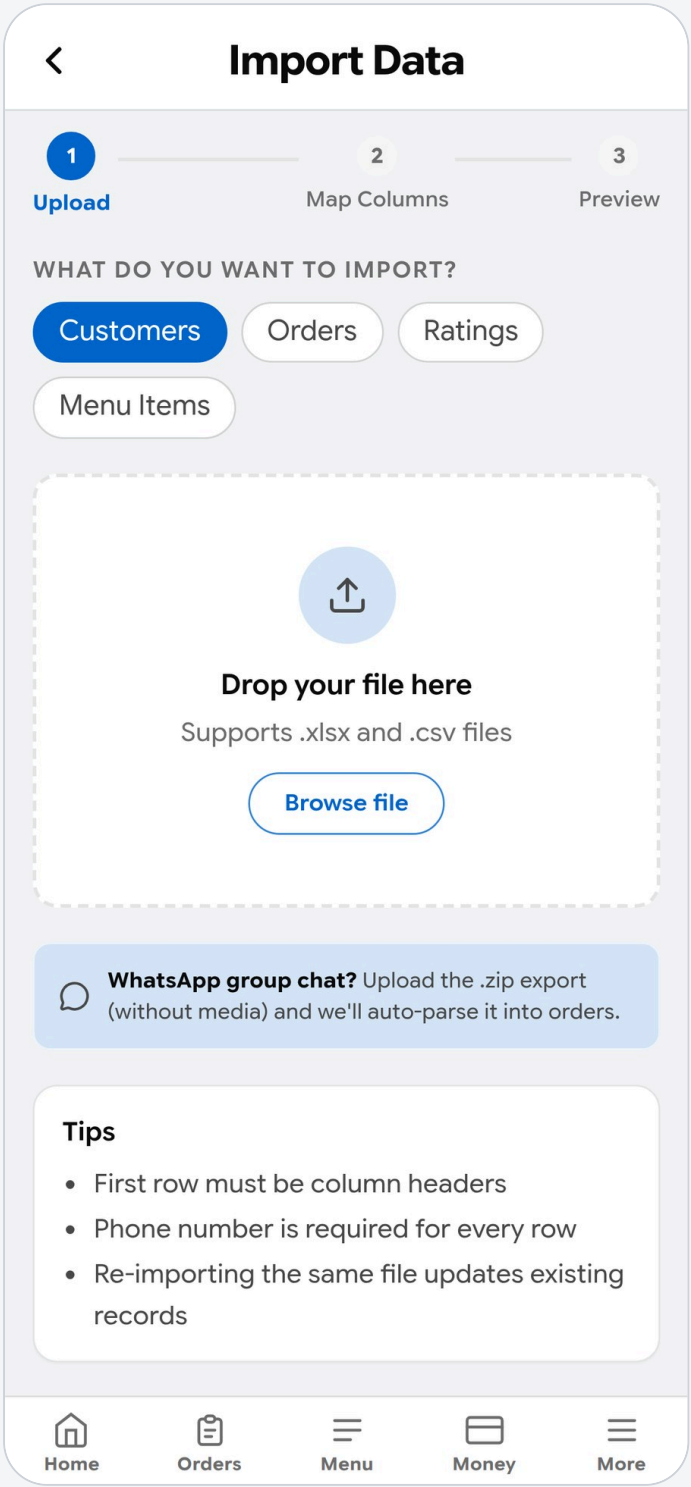
Customers

Every customer's address, order history and dues, built up automatically as orders come in.



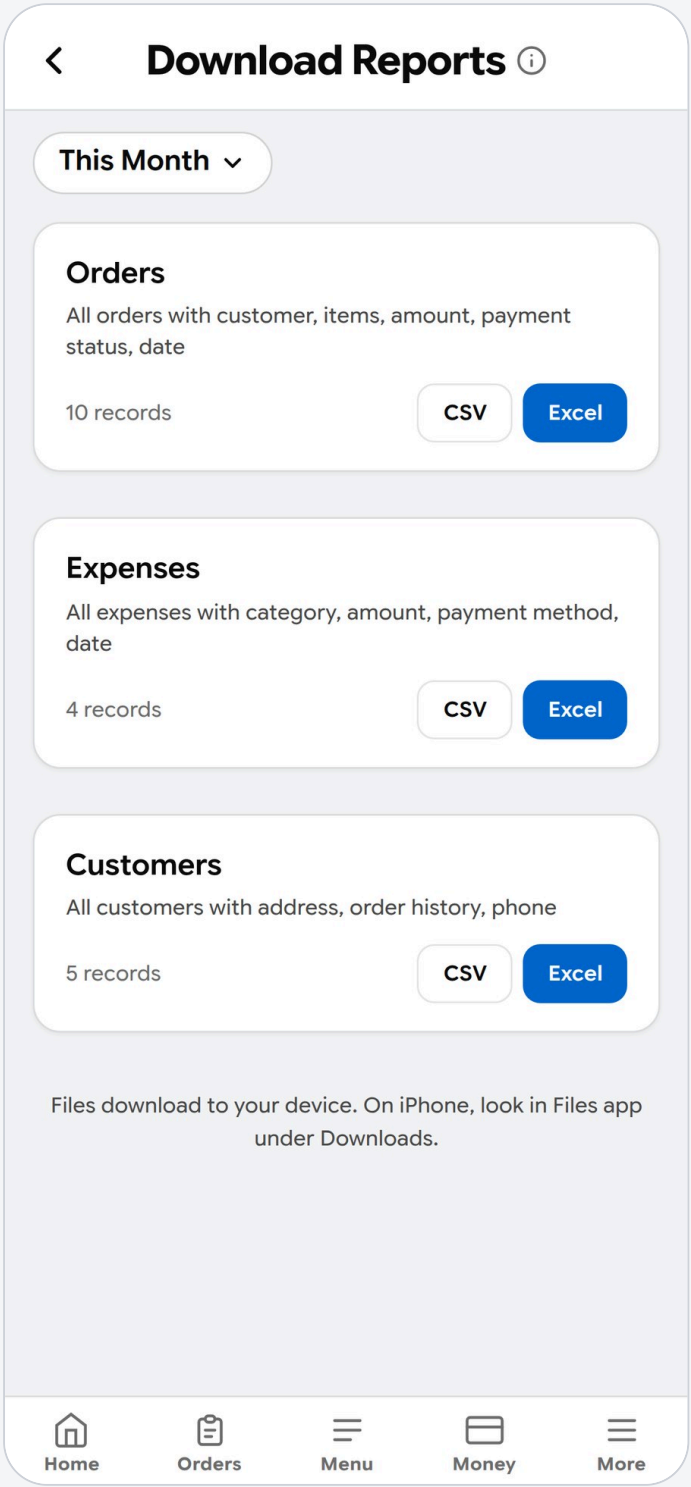
Import

Upload a spreadsheet, or export a WhatsApp chat and have it parsed into orders and customers. The book starts full.



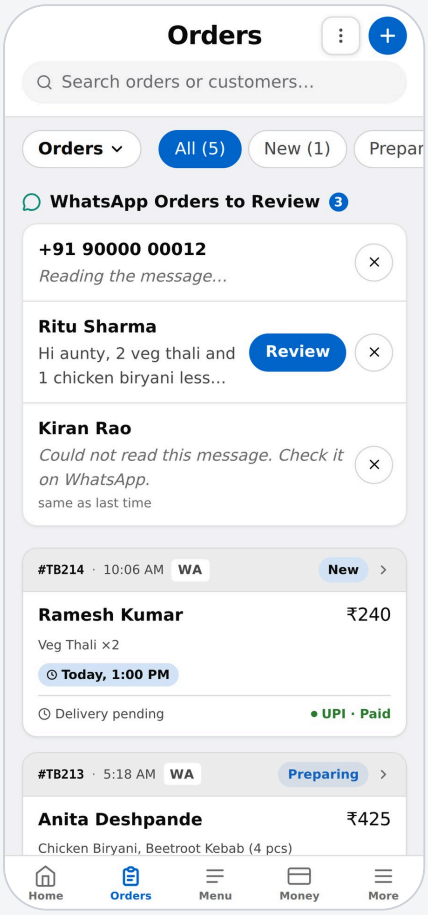
Reports

Orders, expenses and customers exported to CSV or Excel. The owner's data stays the owner's.

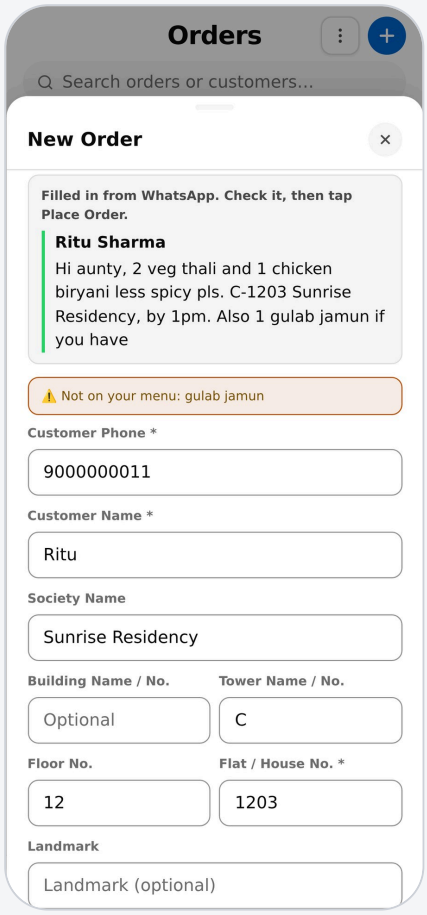


Orders straight from WhatsApp IN TESTING

A message to the kitchen's WhatsApp Business number arrives already read. The owner checks it and taps Place Order. Nothing is ever placed automatically.



Waiting for review



Filled in, ready to check

- The WhatsApp Cloud API delivers each message to a Cloud Function that accepts it only with a valid Meta signature.
- Claude reads it against the kitchen's own menu: dishes, quantities, notes like "less spicy", an address like "C-1203". Anything not on the menu is flagged, never guessed.
- Menu items are checked server-side, quantities clamped, and AI reads capped per kitchen per day.

Status: the app side is built and tested. The WhatsApp connection and AI reading are deployed but have not yet processed a real message. Screens use demo messages.

Under the hood

Frontend	React 19 · TypeScript · Vite · Tailwind CSS 4 · installable PWA
Backend	Firebase Auth (phone OTP) · Cloud Firestore · 10 Cloud Functions in *asia-south1*
Voice	Google Cloud Text-to-Speech, WaveNet voices matched to each language
Payments	UPI deep links and generated QR codes
Languages	English · Hindi · Kannada, with an automated coverage audit
CI/CD	GitHub Actions: 72-screen visual regression on every change, automated function deploys
In testing	WhatsApp Cloud API webhook · Anthropic Claude API, server-side only

Decisions that matter

- **Public writes are server-only.** The storefront never writes to the database; every order goes through a Cloud Function, and the rules deny public writes outright.
- **Default deny, then grant narrowly.** Billing tier is server-only; customer data cannot be listed by anyone but the owner.
- **Costs fail closed.** Paid voice is gated by App Check, sign-in, a plan check and a character cap; if the check errors, voice is denied rather than given away. AI reads are capped per kitchen per day.
- **Every change is screenshot-tested.** Each pull request rebuilds the app against emulators with a fixed demo kitchen and reports which of 72 screens changed.

~210 TypeScript files · ~47,000 lines · 42 screens · ~425 commits

My role

I am not a professional developer. I defined the product, directed an AI-assisted build end to end, tested it on real devices, owned security, and run it as a live product for a real home kitchen. The skill on show is product judgment and the discipline to insist things are actually finished.

Try the live app
ordersin.co/app

Product site
ordersin.co

Case study
github.com/nrnitinrawal-2608/ordersin